

Java Programming

Why another programming language? Don't we have too many already?

Why Java?

What are the differences or advantages of Java?

A Short History of Java

1991: Sun Microsystems engineers, led by Patrick Naughton and James Gosling, a language for consumer devices, the Green project.

C++, Oak, Java

1992: “*7”, an intelligent remote control. Then a cable TV box, Green became “First Person, Inc.” Dissolved 1994.

1994: World wide web, Mosaic, Marc Andressen, University of Illinois; Netscape. HotJava running inside web browsers.

1996: Java 1.0

1997: 1.1 Inner classes

1998: 1.2 strictfp

2000: 1.3

2004: 5.0 Generic classes, “for each” loop, varargs,

autoboxing, metadata, enumerations,
static import

2006: 6

2011: 7

Switch with strings, diamond operator,
binary literals, exception handling,
enhancements

2014: 8

Lambda expressions, interfaces with default
methods, stream and date/time libraries

2017: 9

Modules, miscellaneous language and library
enhancements

2018: 11

Local variable type inference (`var`), HTTP
client, removal of Java FX, JNLP, Java EE
overlap, and CORBA

2021: 17

Today!

Switch expressions, text blocks, instanceof
pattern matching, records, sealed classes

Characteristics of the Java Programming Language

Simple

Object-Oriented

Architecture-Neutral

Portable

Interpreted

Multithreaded

Secure

Robust

High-Performance

Distributed

Dynamic

Common Misconceptions about Java

Is an extension of HTML

XML can do the same

Is easy to learn

Will become a universal language

Is just another programming language

Is proprietary

Is slow because interpreted

Program runs inside web page

Has security risk

More complex than JavaScript

Able to replace desktop with “Internet appliance”

Books

Required:

Core Java, Volume 1: Fundamentals, 12th edition.
Horstmann, Cay. Oracle Press, December 2021. ISBN:
978-0137673629.

Recommended:

-*Java Cookbook, Problems and Solutions for Java Developers*, fourth edition, Ian F. Darwin, O'Reilly, 2020,
ISBN: 978-1492072584

-*Java in a Nutshell*, a Desktop Quick Reference, seventh edition, Benjamin J. Evans and David Flanagan, O'Reilly, 2019, ISBN 978-149203725

-*Data Structures & Algorithm Analysis in Java*, third edition, Clifford A. Shaffer, Dover Publications, 2011, ISBN 978-0486485812

The Java Programming Environment

Installing the Java Development Kit

Downloading the JDK, Standard Edition:

oracle.com/javadownload

OpenJDK

jdk.java.net

Installing the software:

Mac: /Library/Java/JavaVirtualMachine/
jdk-17.0.x.jdk/Contents/Home

Windows: E.g., c:\Java\jdk-17.0.x

From now on, we'll refer to our installation directory
as *jdk*

Testing the installation:

On a terminal window:

`javac --version`

Output:

`javac 17.0.4`

Additional setting for Windows: Book, p. 19

Installing Source Files and Documentation

-Source Files:

Unpacking compressed file `jdk/lib/src.zip`:

Make a directory in home:

`mkdir javasrc`

Locate `src.zip` and unzip it into `javasrc` directory:

`cd javasrc`

`jar xvf jdk/lib/src.zip`

-Documentation:

Download from

www.oracle.com/technetwork/java/javase/downloads

The download documentation file is

`jdk-17.0.x_doc-all.zip`

Unzip it:

`jar xvf Downloads/jdk-17.0.x_doc-all.zip`

Using the Command-Line Tools

-To compile the Helloworld class in Helloworld.java:

`javac Helloworld.java`

-To run the compile Helloworld Java class:

`java Helloworld`

Using an Integrated Development Environment

Eclipse, IntelliJ IDEA, NetBeans

Download Eclipse: eclipse.org/downloads

Writing a program in Eclipse:

File, New, Project, Java Project

Navigate to the project Location, Finish

On left pane, Double-click on <filename>.java

Click on project name, Run, Run-As, Java Application

Program output is on bottom consol pane

Errors, if any, are shown on error pane below source code

Fundamental Programming Structures

Comments

```
/* */  
//
```

Data Types

integer, floating-point, char, boolean

Data Type: Integer

byte	1 byte	-128 to 127
short	2	-32768 to 32767
int	4	-2,147,483,648 to 2,147,483,647
long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Data Type: Floating Point

float 4 +-3.40282347 x 10E38

double 8 +-1.79769313486231750 x 10E308

Suffix for float: F or f, e.g., 3.14F

Without suffix, default is double

Suffix for exponent: E, e.g., 1.483E4 is 14830

In hexadecimal, using p for exponent, because
“E” is a hexadecimal digit

Special floating-point values:

Positive infinity

Double.POSITIVE_INFINITY

E.g., number/o

Negative infinity

Double.NEGATIVE_INFINITY

NaN, Not a Number

Double.NaN

E.g., 0/0, square root of negative

Caution of round-off errors. E.g., $2.0 - 1.1$ gives 0.89999...

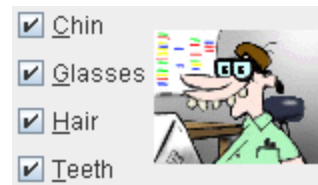
A Visual Guide to Swing Components

Basic Controls

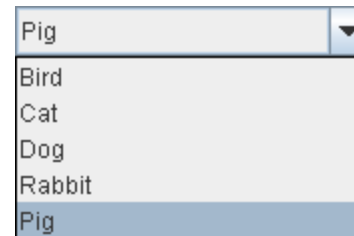
Simple components that are used primarily to get input from the user; they may also show simple state.



[JButton](#)



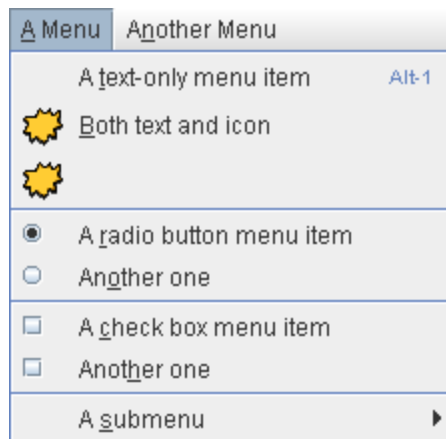
[JCheckBox](#)



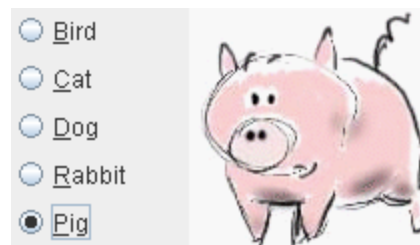
[JComboBox](#)



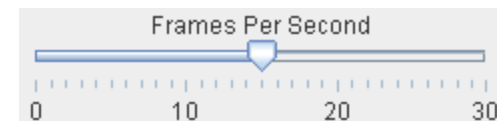
[JList](#)



[JMenu](#)



[JRadioButton](#)



[JSlider](#)



[JSpinner](#)



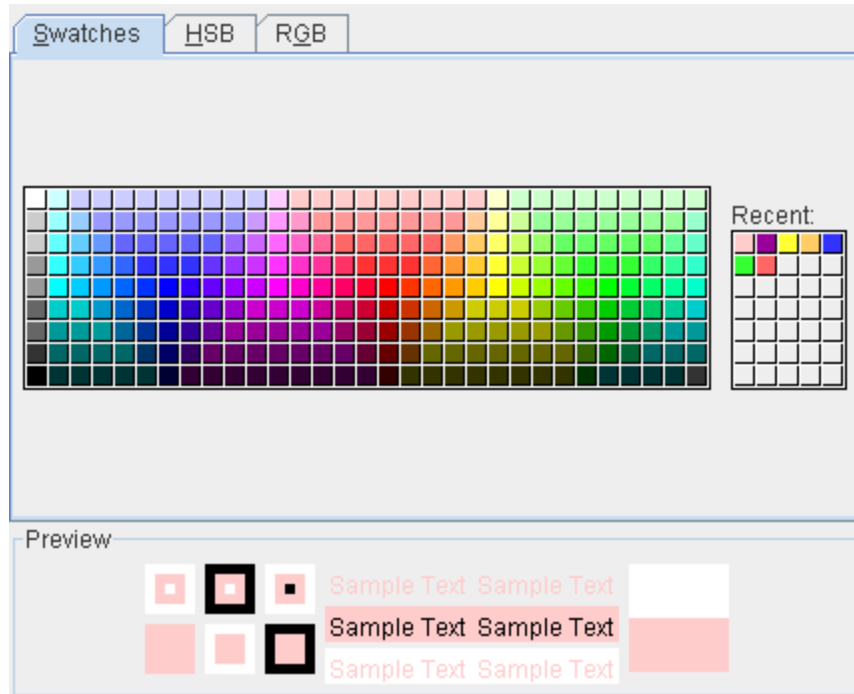
[JTextField](#)



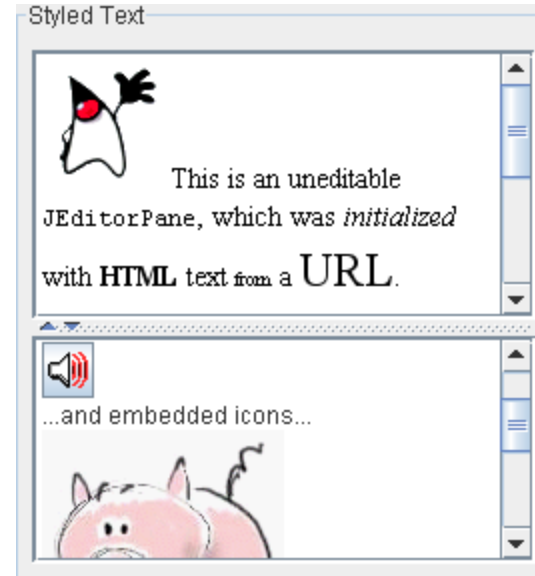
[JPasswordField](#)

Interactive Displays of Highly Formatted Information

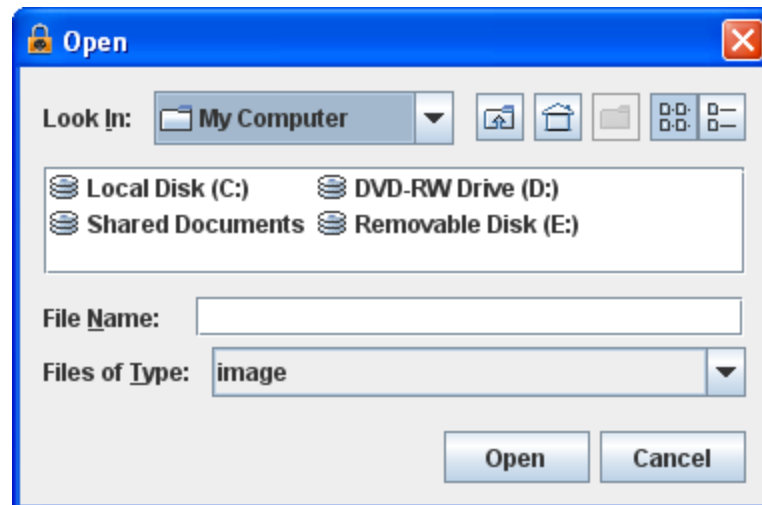
These components display highly formatted information that (if you choose) can be modified by the user.



[JColorChooser](#)



[JEditorPane](#) and [JTextPane](#)



[JFileChooser](#)

Host	User	Password	Last Modified
Biocca Games	Freddy	!#asf6Awwzb	Mar 16, 2006
zabble	ichabod	Tazb!34\$fZ	Mar 6, 2006
Sun Developer	fraz@hotmail.co...	AasW541!fbZ	Feb 22, 2006
Heirloom Seeds	shams@gmail....	bkz[ADF78!	Jul 29, 2005
Pacific Zoo Shop	seal@hotmail.c...	vbAf124%z	Feb 22, 2006

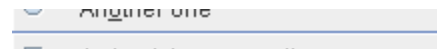
[JTable](#)

This is an editable JTextArea. A text area is a "plain" text component, which means that although it can display text in any font, all of the text is in the same font.

[JTextArea](#)[JTree](#)

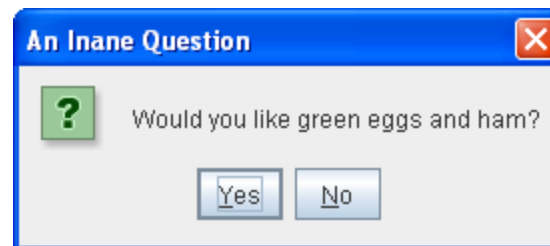
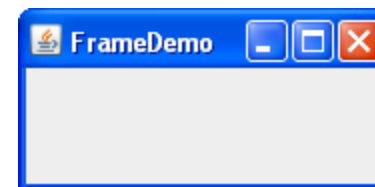
Uneditable Information Displays

These components exist solely to give the user information.

[JLabel](#)[JProgressBar](#)[JSeparator](#)[JToolTip](#)

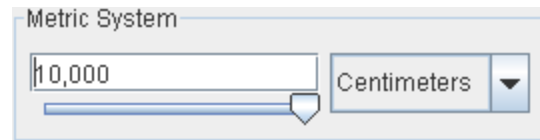
Top-Level Containers

At least one of these components must be present in any Swing application.

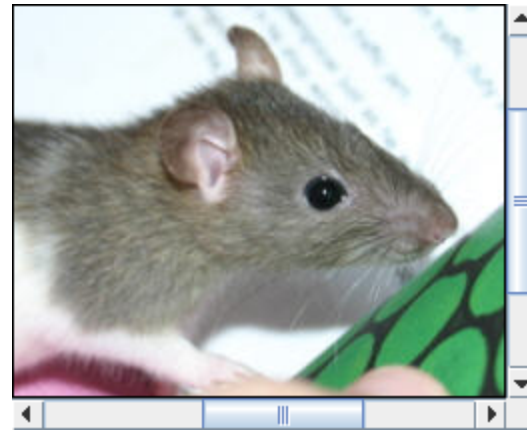
[JApplet](#)[JDialog](#)[JFrame](#)

General-Purpose Containers

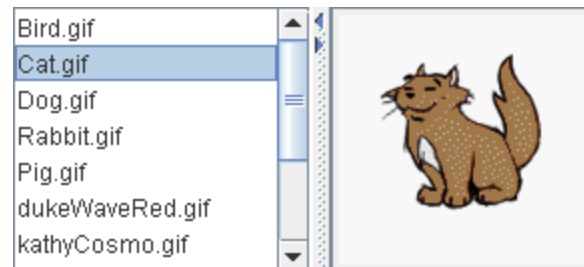
These general-purpose containers are used in most Swing applications.



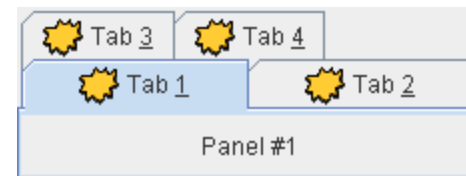
[JPanel](#)



[JScrollPane](#)



[JSplitPane](#)



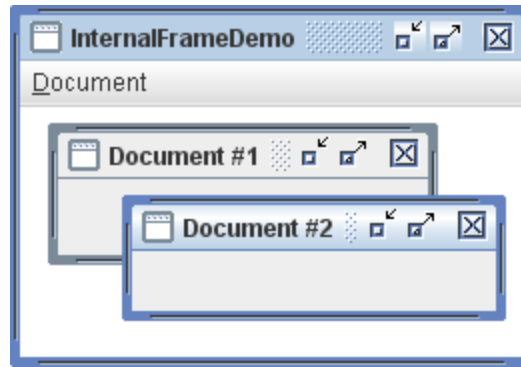
[JTabbedPane](#)



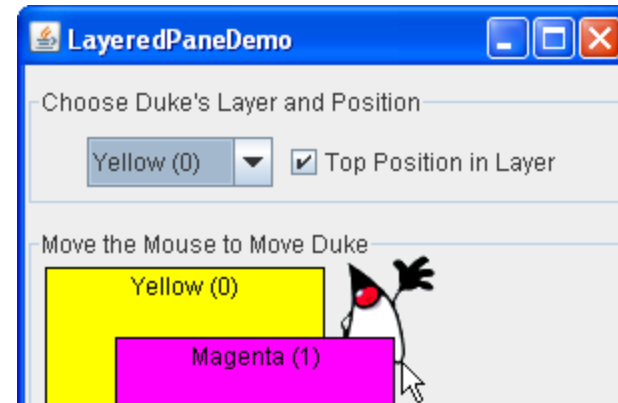
[JToolBar](#)

Special-Purpose Containers

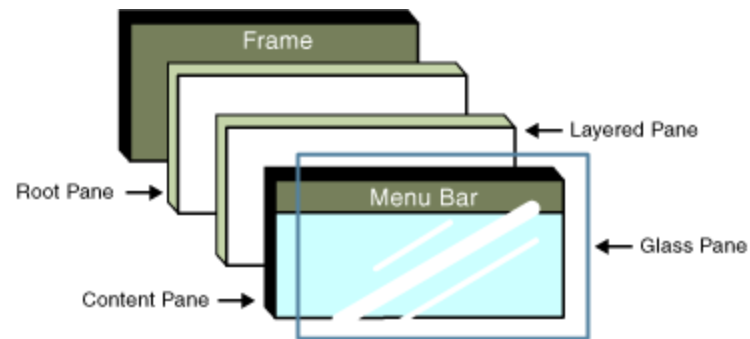
These special-purpose containers play specific roles in the UI.



[JInternalFrame](#)



[JLayeredPane](#)



[Root pane](#)

Your use of this page and all the material on pages under "The Java Tutorials" banner, and all the material on pages under "The Java Tutorials" banner is subject to the Java SE Tutorial Copyright and License. Additionally, any example code contained in any of these Java Tutorials pages is licensed under the Code Sample License.

Essential Java Knowledge for the Password Verifier Project

A guide to the Java concepts needed to build a password verification program.

Introduction to Java Programming

Overview of
Java and its
applications.

Why Java is
suitable for
this project.

Basic Java Syntax and Structure

Understanding
the anatomy of a
Java program.

Introduction to
classes, methods,
and the main
method.

Setting Up the Java Development Environment

Steps to
install JDK and
set up an IDE.

Compiling and
running Java
programs.

Input and Output in Java

01

Using the Scanner class
for reading input.

02

Displaying output
using
`System.out.println()`.

Control Flow Statements

If-else statements
for making
decisions.

Looping
constructs (for
and while loops).

Working with Strings

String basics and
common
methods.

Checking the
length and
accessing
characters.

Character Methods in Java

Overview of the
Character class.

Using
`isUpperCase()`,
`isLowerCase()`, and
`isDigit()` methods.

Defining and Using Methods

Creating methods
to encapsulate
functionality.

Parameters,
return types, and
calling methods.



Error Handling Basics

Introduction to try-catch blocks for basic error handling.

Putting It All Together

Combining all the concepts to build the Password Verifier program.

Overview of how each piece fits into the project.

Conclusion and Next Steps

Summary of
key learnings.

Resources for
further study
and practice.

Essential Java Knowledge for Weekly Sales Analysis Project

A guide to the Java concepts needed to build a weekly sales analysis program.

Introduction to Java Programming

Overview of Java
and why it's used for
data analysis tasks.

Brief explanation of
Java's role in
handling file I/O and
data processing.

Setting Up Java Development Environment

1

Installing JDK and setting up an IDE.

2

Compiling and running Java programs from the IDE or command line.

Basic Java Syntax and Structure

- Understanding classes, methods, and the main method.
- Example Code:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

File Handling in Java

- Reading from files using FileReader and BufferedReader.
- Handling exceptions with try-catch blocks.
- Example Code:

```
try (BufferedReader br = new BufferedReader(new
FileReader("sales_data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Working with Arrays and Lists

- Using arrays and lists to store data.
- Adding data to lists and iterating over elements.
- Example Code:

```
List<Double> sales = new ArrayList<>();  
sales.add(100.5);  
sales.add(200.75);  
for (double sale : sales) {  
    System.out.println(sale);  
}
```

Control Flow Statements

- Using loops (for, while) to process data.
- Using if-else statements for decision making.
- Example Code:

```
for (int i = 0; i < 5; i++) {  
    if (i % 2 == 0) {  
        System.out.println("Even number: " + i);  
    } else {  
        System.out.println("Odd number: " + i);  
    }  
}
```

Calculations and Aggregations

- Performing arithmetic operations for totals and averages.
- Calculating sums and averages from arrays or lists.
- Example Code:

```
double[] sales = {100.5, 200.75, 150.25};  
double total = 0;  
for (double sale : sales) {  
    total += sale;  
}  
double average = total / sales.length;  
System.out.println("Total: " + total + ", Average: " + average);
```

Tracking Maximum and Minimum Values

- Identifying the highest and lowest values in a dataset.
- Using variables to track these values as data is processed.
- Example Code:

```
double max = Double.MIN_VALUE;  
double min = Double.MAX_VALUE;  
for (double sale : sales) {  
    if (sale > max) max = sale;  
    if (sale < min) min = sale;  
}  
System.out.println("Max: " + max + ", Min: " + min);
```

Putting It All Together

- Integrating all the concepts into the Weekly Sales Analysis program.
- Overview of how each concept contributes to the final implementation.

Conclusion and Next Steps

Summary of key
concepts.

Additional resources
for learning more
about Java and data
processing.